Global Disaster Data Pipeline

Layered flow: five free APIs > hybrid ingestion > Kafka > Spark ETL > PostgreSQL > dashboard

01 SOURCES — Upstream APIs

- **USGS** (GeoJSON) — Earthquake events
- **GDACS** (GeoJSON) — Multi-hazard alerts
- **NASA FIRMS** (CSV) — Fire detections
- **NASA EONET** (JSON) — Curated events
- **OPEN-METEO** (JSON) — Weather forecasts

02 INGESTION — Python + Airflow

- **Python continuous pollers**
  USGS every 1 min | GDACS every 10 min
  source IDs + update detection
- **Apache Airflow: scheduled ingestion DAGs**
  FIRMS / 2 h EONET / 1 h Open-Meteo / 3 h
  retry + checkpoints + historical backfills

archive

03 TRANSPORT — Kafka + archive

- **Apache Kafka**
  raw.usgs | raw.gdacs | raw.firms | raw.eonet | raw.openmeteo
  record-level messages • stable message keys • at-least-once consumers
- **MinIO: raw archive**
  Original API responses
  Reprocess / audit

04 STAGING — Kafka consumer

- **Python consumer**
  validate envelope + upsert
- **PostgreSQL staging**
  staging.source_records (JSONB) + ops.ingestion_runs / checkpoints

05 ETL / MODEL — Airflow + Spark

- **Apache Spark batch ETL**
  normalize • deduplicate • aggregate • join
  Airflow triggers Spark every 15 min
- **PostgreSQL final schema + analytics**
  7 core tables + daily counts, hotspot grids, active alerts

06 SERVING — FastAPI

- **FastAPI**
  GET /api/v1/events /fires /weather /analytics
- **Dashboard / UI**
  maps • filters • KPIs • charts

What runs, when, and what it writes

Concrete service responsibilities, schedules, Kafka topics, ETL checkpoints, and recovery behavior

01 INGEST — Poll / archive / publish

→

02 BUFFER + STAGE — Kafka consumer / PostgreSQL

→

03 TRANSFORM + SERVE — Airflow / Spark / FastAPI

SOURCE	RUNNER	POLL	KAFKA TOPIC	DEDUPE / VERSION KEY	OUTPUT INTENT
USGS	Python poller	1 min	raw.usgs	USGS event id + updated	new / revised earthquakes
GDACS	Python poller	10 min	raw.gdacs	type + event id + alert revision	alert history and event updates
NASA FIRMS	Airflow DAG	2 h	raw.firms	sensor + time + pixel location	satellite detections, not confirmed fires
NASA EONET	Airflow DAG	1 h	raw.eonet	EONET event id + revision	open / recently closed event changes
Open-Meteo	Airflow DAG	3 h	raw.openmeteo	site + retrieval + forecast hour	forecast snapshots at event sites

Airflow DAGs and dependencies

- **ingest_firms / ingest_eonet**
  fetch > raw MinIO object > changed records to Kafka
- **ingest_weather**
  read active event locations > site list > forecast API
- **transform_and_publish**
  read staging > Spark > PostgreSQL upserts
- **reconcile_daily**
  overlap/backfill > checks > replay missing records

Processing and correctness rules

- **Archive before publish**
  MinIO keeps original responses + metadata.
- **Publish changes, not every repeated poll**
  Use stable record IDs and source update timestamps.
- **Commit after staging write**
  Kafka consumer can replay safely with unique constraints.
- **Spark loads incrementally**
  Watermark + overlap; temp tables then SQL upsert.

DEPLOYMENT — **Docker Compose: kafka | airflow | spark | minio | postgres | python-producers | kafka-consumer | fastapi**

Weather sampling reads current event locations only after an ETL cycle; API serves PostgreSQL, never upstream sources.

Raw source inventory

Representative upstream fields and the canonical transformations each feed contributes; selected fields only

01 USGS EARTHQUAKES

GeoJSON | 1-min polling

FIELDS AVAILABLE	id, properties.mag, magType, place, time, updated, status, tsunami, geometry.coordinates [lon,lat,depth]
TRANSFORM	Event ID; UTC times; magnitude/depth; revised-event upsert.

02 GDACS MULTI-HAZARD

GeoJSON | 10-min polling

FIELDS AVAILABLE	event id/type, event dates, alert level, episode information, location/geometry, event description; impact estimates where available
TRANSFORM	Canonical hazard type; alert-level history; record event/episode revisions.

03 NASA FIRMS (VIIRS NOAA-20 NRT)

CSV | every 2 hours

FIELDS AVAILABLE	latitude, longitude, bright_ti4, bright_ti5, scan, track, acq_date, acq_time, satellite, instrument, confidence, frp, daynight
TRANSFORM	Parsed UTC acquisition; detection fingerprint; FRP stats; daily lat/lon-grid aggregation.

04 NASA EONET

JSON | every hour

FIELDS AVAILABLE	id, title, description, categories, sources, closed, geometry[]: date + Point/Polygon, optional magnitude metadata
TRANSFORM	Map categories; retain open/closed status and date-stamped geometry history.

05 OPEN-METEO

JSON | every 3 hours

FIELDS AVAILABLE	latitude, longitude, hourly.time, temperature_2m, relative_humidity_2m, precipitation, wind_speed_10m, wind_gusts_10m
TRANSFORM	Sample at chosen event sites; unnest hourly arrays; preserve forecast retrieval time.

SOURCE NOTES

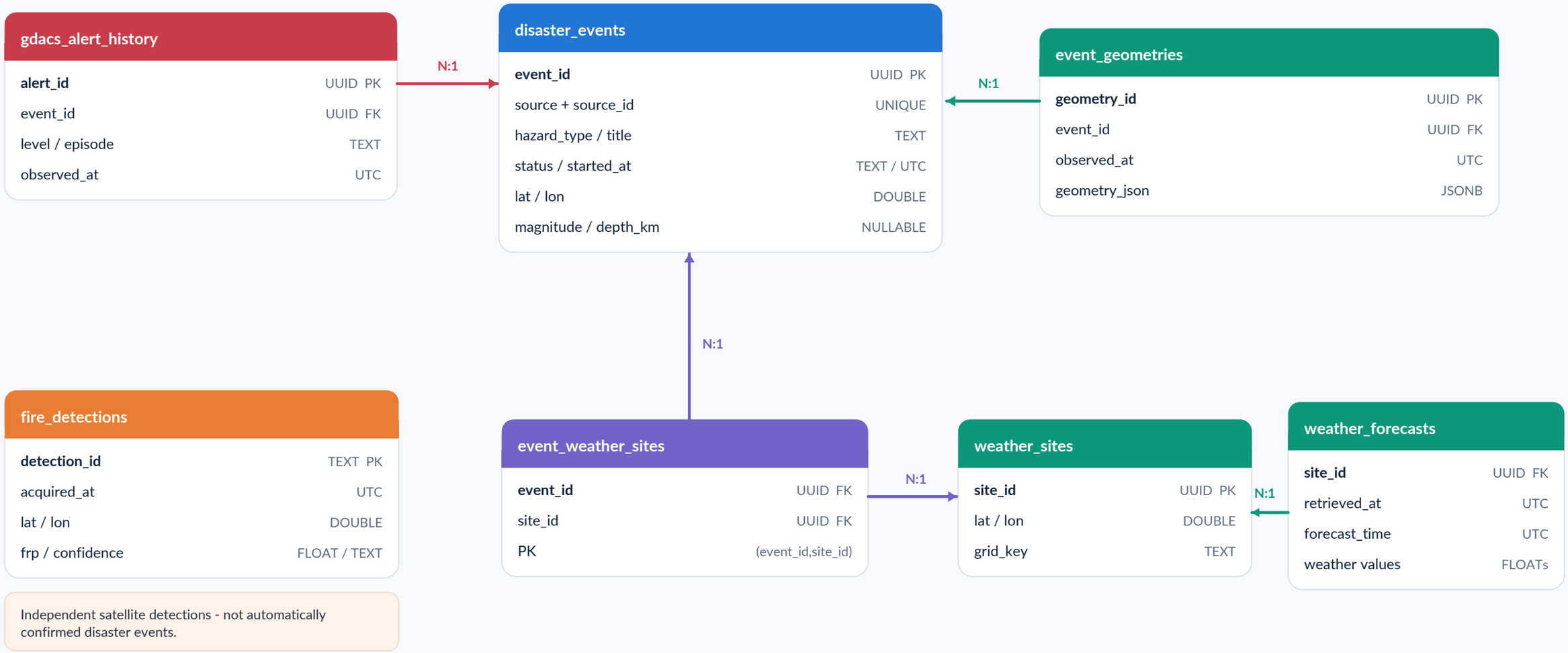
USGS records can be revised; GDACS alert levels can change; FIRMS detections are thermal anomalies rather than confirmed wildfire incidents. EONET is curated and may report the same underlying event as other sources. Open-Meteo is forecast enrichment, not a disaster catalog.

Official API documentation: click the format / frequency label in each inventory card.

Minimal PostgreSQL database design

Source-native events remain separate; 7 core tables, one generic staging table, and summary views

APPENDIX B · ERD



STAGING

staging.source_records

source, native_id, payload JSONB, hash, ingested_at, processed_at

ANALYTICS (SPARK / SQL)

daily_event_counts • daily_fire_grid • active_alerts

latest_event_weather • source_freshness

MODEL RULE

Foreign keys reference source-native events; cross-source matching is deferred. Store locations as lat/lon plus JSONB.

Proposed FastAPI endpoint catalog

Versioned REST API reading PostgreSQL core tables and precomputed analytics only

METHOD	ENDPOINT	DASHBOARD PURPOSE	READS
GET	/api/v1/events	List events; filters: source, hazard_type, date, bbox	disaster_events
GET	/api/v1/events/{event_id}	Single event with alerts and geometry history	events + alerts + geometries
GET	/api/v1/earthquakes	Filter earthquakes by minimum magnitude and depth	disaster_events
GET	/api/v1/alerts/active	Current GDACS alerts, optionally filtered by level	active_alerts
GET	/api/v1/fires/heatmap	Grid-based daily fire intensity for maps	daily_fire_grid
GET	/api/v1/fires/detections	Paginated hotspot points in a bbox/date window	fire_detections
GET	/api/v1/weather	Weather forecasts for a site or a linked event	weather_forecasts + sites
GET	/api/v1/analytics/daily	Counts and trends by date, source, hazard, country	daily_event_counts
GET	/api/v1/system/freshness	Most recent ingest time, status and processing lag	source_freshness

QUERY & RESPONSE CONVENTIONS

Common filters: from, to, bbox=west,south,east,north, limit, offset, hazard_type, source. Times: UTC ISO-8601. Large data: paginated results or aggregated grids. Format: JSON.

RECOMMENDED BUILD ORDER

1 Docker + PostgreSQL + MinIO 2 Kafka topics/consumer 3 USGS & GDACS producers 4 Airflow batch sources 5 Spark ETL + models 6 FastAPI 7 dashboard + tests.

DESIGN BOUNDARY

No Prefect, dbt, PostGIS, H3, or dedicated Spark Structured Streaming requirement. Technical scope follows the supplied DSAI5102 Course Section Guide (weeks 5, 8, 10 and 11). The source-field inventory is based on public API documentation; implementation schedules and schema are project proposals.